

# CoderCommands MCP System

Step 01 Record — Repository Scaffolding

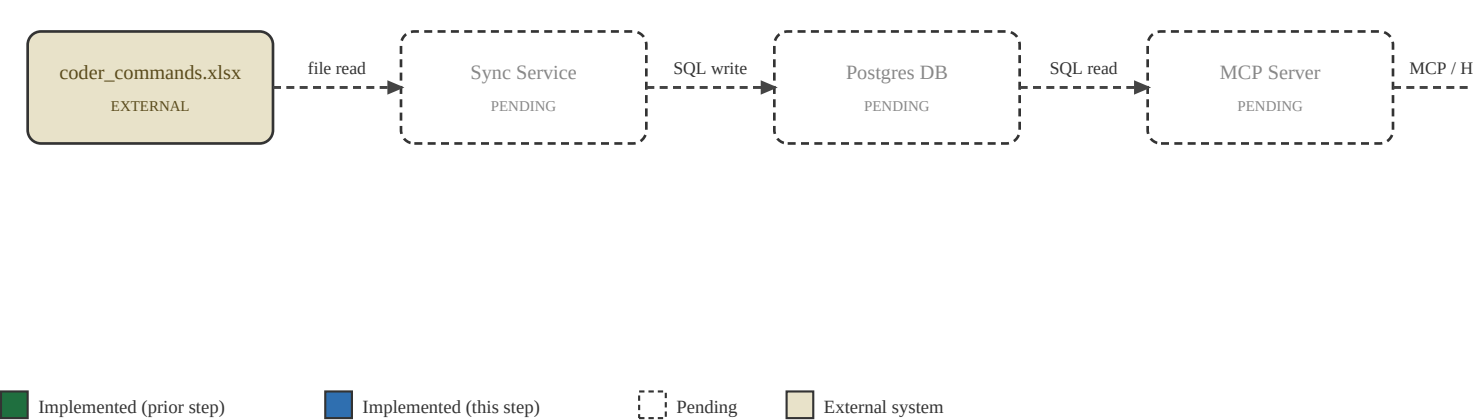
|          |                                            |
|----------|--------------------------------------------|
| DOCUMENT | CoderCommands MCP System — Step Record     |
| SYSTEM   | CoderCommands MCP Command-Reference Server |
| STEP     | 01 — Repository Scaffolding                |
| DATE     | 2026-08-15                                 |
| REVISION | 1.0                                        |
| STATUS   | Complete                                   |

## 1. Purpose & Scope

Establish the fixed directory layout and environment-configuration surface that every later step builds into. This is the equivalent of laying out cabinet/panel space and terminal numbering before any device is mounted: nothing here is functional yet, but every subsequent component has a defined, unambiguous location and naming convention.

## 2. System Context Diagram

Signal-flow view of the four functional units (Sync Service, Postgres, MCP Server, FastAPI Gateway) plus the two external endpoints (source workbook, Claude Code client). Fill/border indicates build state as of this step.



## 3. File / Component Register

Role of every file or component introduced or modified in this step, with its upstream and downstream interfaces (I/O-list style).

| Ref | File / Component | Role                                                             | Interface In   | Interface Out      |
|-----|------------------|------------------------------------------------------------------|----------------|--------------------|
| R1  | db/              | Directory reserved for the Postgres schema definition (Step 2).  | —              | —                  |
| R2  | sync/            | Directory reserved for the ingest + watcher service (Steps 3–4). | xlsx (planned) | Postgres (planned) |

| Ref | File / Component    | Role                                                                                                                           | Interface In         | Interface Out          |
|-----|---------------------|--------------------------------------------------------------------------------------------------------------------------------|----------------------|------------------------|
| R3  | mcp_server/         | Directory reserved for the MCP tool server (Step 5).                                                                           | Postgres (planned)   | FastAPI (planned)      |
| R4  | api/                | Directory reserved for the FastAPI gateway (Step 6).                                                                           | MCP server (planned) | Claude Code (planned)  |
| R5  | .env.example        | Template for all runtime configuration (DB credentials, ports, poll interval, xlsx path). Copied to .env, which is gitignored. | —                    | All services (planned) |
| R6  | .gitignore          | Excludes .env, Python caches, and any local Postgres data volume from version control.                                         | —                    | —                      |
| R7  | coderCommands_Plan/ | Output directory for this series of step records.                                                                              | —                    | —                      |

## 4. Work Performed

- Created four functional-unit directories at the project root: db/, sync/, mcp\_server/, api/ — one per component in the target system context diagram.
- Created coderCommands\_Plan/ as the fixed output location for step records.
- Added .env.example as the single source of configurable parameters (credentials, ports, xlsx path, sync poll interval), so no service will hard-code configuration in Step 2 onward.
- Added .gitignore to keep secrets (.env) and local build artifacts out of version control ahead of any future git init.

## 5. Issues Encountered & Resolution

| Issue                                                                                                                                        | Root Cause                                                                                                                                   | Resolution                                                                                                                                                                                                                                               |
|----------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Host machine already runs an unrelated Postgres container (name: postgres, publishing host port 5432) from a separate project (mcp-gateway). | Two independent Docker Compose projects on the same host will collide on container name and published port if both default to postgres/5432. | Reserved POSTGRES_PORT in .env.example for internal container use only; the compose network (Step 7) will not publish Postgres to the host, and service/container names will be namespaced (codercommands-*) to avoid collision with the existing stack. |

## 6. Verification Record

| Check                                       | Method                                 | Result        |
|---------------------------------------------|----------------------------------------|---------------|
| Four functional-unit directories exist      | Is of project root                     | PASS          |
| coderCommands_Plan/ exists and is writable  | PDF written to that path by this build | PASS          |
| .env.example present with all required keys | Manual review                          | PASS (manual) |

## 7. Next Step / Open Dependencies

Step 2 defines the Postgres schema (db/schema.sql) that the sync service (Step 3–4) will write into and the MCP server (Step 5) will read from.